

「README」

TinyCRM TW

為台灣中小型業務量身打造的迷你 CRM — 手機優先、LINE 備註、一鍵匯出 Excel，內建藍新金流訂閱與 AI 業務筆記摘要。

狀態：開發中（demo 模式優先）。前端、API、金流加解密邏輯與法務頁均已實作；資料層目前以 demo 種子 / in-memory 運作，Prisma schema 已備妥待接 PostgreSQL。

專案簡介

TinyCRM TW 是一套輕量級客戶關係管理工具，鎖定業務、房仲、保險、接案者與教育招生等需要隨手記錄客戶與跟進的場景。介面以手機優先設計，支援聯絡人管理、互動歷史時間軸、Excel 匯出，並整合藍新金流（NewebPay）訂閱付款與透過 Smart Router 的 AI 業務筆記摘要。

目前資料層以內建 demo 種子資料運作，方便在未連接資料庫前直接試用；Prisma schema 已備妥（`Contact` / `Interaction` / `Order` / `Subscription`），待設定 `DATABASE_URL` 後即可接上 PostgreSQL 持久化。

專案文件

文件	說明
docs/ARCHITECTURE.md	專案總覽、技術棧、架構圖與 DB 實體關係圖（Prisma schema）
docs/FLOWS.md	6 個關鍵流程的圖解（列表 / 新增 / 互動 / 匯出 / AI 摘要 / 金流）
docs/PAGES.md	頁面路由表 + API endpoint 表 + 共用模組與方案定價
docs/OPERATIONS.md	安裝 / 啟動指令、環境變數清單、部署與維運操作

核心功能

依實際程式碼與路由實作：

- 聯絡人列表與統計（`/`）：全部 / 潛在客戶 / 已洽談 / 現有客戶 計數卡片 + RWD 表格，顯示姓名、公司、狀態徽章、標籤。
- 新增聯絡人（`/contacts/new`）：表單收集姓名、電話、Email、LINE ID、公司、標籤、狀態、備註，送出至 `POST /api/contacts`。
- 聯絡人詳細頁與互動時間軸（`/contacts/[id]`）：顯示基本欄位、備註，與依時間排序的互動歷史；可透過 Server Action（`addInteractionAction`）新增互動（電話 / 訊息 / 會議 / 筆記 / Email / LINE）。
- Excel 匯出（`GET /api/contacts/export`）：以 `xlsx` 產生 `.xlsx`，欄位含姓名、電話、Email、LINE ID、公司、狀態、標籤、備註、來源、建立時間。
- AI 業務筆記摘要（`POST /api/lead-summary`）：將業務自由筆記壓縮成「一句話摘要 + 具體下一步」，經由本機 Smart Router 免費層呼叫 LLM，避免消耗付費額度。
- 藍新金流訂閱付款（NewebPay MPG）：
 - `POST /api/payment/newebpay/checkout`：依方案（`basic` / `pro`）建立訂單並產生 MPG 加密參數（AES-256-CBC + SHA256 TradeSha）。
 - `POST /api/payment/newebpay/notify`：Server-to-server 付款通知，驗證並更新訂單狀態，固定回應 `0`。

- `POST /api/payment/newebpay/return`：使用者端導回，解碼訂單編號後 303 轉址至付款結果頁 `/payment/success`。
- 方案定價（`lib/payment/pricing.ts`）：基礎版 NT\$499/月（50 位客戶上限）、進階版 NT\$899/月（無限客戶 + SMS 提醒 + 消費分析）。
- 法務頁與付款結果頁：`/pricing`、`/terms`、`/privacy`、`/refund`、`/payment/success`（公司資訊單一來源於 `lib/company.ts`）。

備註：目前 `/api/contacts` 與互動操作為 demo / 記憶體模式（回傳成功但尚未寫入資料庫），聯絡人 / 互動畫面讀取的是 `lib/seed-data.ts` 的種子資料；訂單與訂閱亦以 in-memory store（`lib/payment/order-store.ts`）暫存。所有持久化待 `DATABASE_URL` 設定並執行 `prisma db push` 後切換為 Prisma Client。

技術棧

類別	技術
框架	Next.js 16.2.4 (App Router)、React 19.2.4
語言	TypeScript 5
樣式	Tailwind CSS v4 (<code>@tailwindcss/postcss</code>)
UI 工具	<code>lucide-react</code> 、 <code>class-variance-authority</code> 、 <code>clsx</code> 、 <code>tailwind-merge</code>
資料庫	Prisma 6 + PostgreSQL (Neon / Supabase 任選)
匯出	<code>xlsx</code> (SheetJS)
金流	藍新金流 NewebPay MPG (Node <code>crypto</code> ，無額外 SDK)
AI	本機 Smart Router (OpenAI 相容 <code>/v1/chat/completions</code>)
Lint	ESLint 9 + <code>eslint-config-next</code>

完整技術棧與版本見 [docs/ARCHITECTURE.md](#)。

快速開始

需求：Node.js 20+。

```
# 1. 安裝相依套件
npm install

# 2. 設定環境變數 (見 docs/OPERATIONS.md)
cp .env.example .env

# 3. (可選) 連接資料庫後同步 schema
npx prisma db push

# 4. 啟動開發伺服器 (http://localhost:3000)
npm run dev
```

其他指令（來自 `package.json` scripts）：

```
npm run build    # 正式建置
npm run start    # 啟動正式伺服器
```

```
npm run lint    # ESLint 檢查
```

不設定資料庫也能啟動：列表 / 詳細 / 匯出 會直接使用 demo 種子資料。

環境變數清單與部署方式詳見 [docs/OPERATIONS.md](#)。

授權

本專案未附授權檔（無 `LICENSE`），預設為 **Private — All rights reserved**。未經授權請勿散布或商用。

「ARCHITECTURE.md」

架構說明 (ARCHITECTURE)

TinyCRM TW — 為台灣中小型業務量身打造的迷你 CRM。手機優先、LINE 備註、一鍵匯出 Excel，內建藍新金流 (NewebPay) 訂閱付款與透過 Smart Router 的 AI 業務筆記摘要。

專案總覽

TinyCRM TW 是一套基於 **Next.js 16 App Router** 的單一 Web 應用，鎖定業務、房仲、保險、接案者與教育招生等需要隨手記錄客戶與跟進的場景。整個專案同時包含：

- 前端頁面 (聯絡人列表 / 新增 / 詳細頁 + 互動時間軸、定價、付款結果、法務頁)。
- 後端 API Route (新增聯絡人、Excel 匯出、AI 筆記摘要、藍新金流 checkout / notify / return)。
- Server Action (在詳細頁新增互動)。

目前資料層狀態 (重要)

實際程式碼為「demo 模式優先」設計，方便在未連接資料庫前直接試用：

- 列表 / 詳細 / 匯出讀取的是 `lib/seed-data.ts` 內建的種子資料 (`seedContacts` / `seedInteractions`)。
- `POST /api/contacts` 僅驗證後回傳 `{ ok: true, demo: true }`，尚未寫入資料庫 (程式碼內標註 `TOD0: persist via Prisma`)。
- 在詳細頁新增互動 (`addInteractionAction`) 是以 `appendInteraction` 推入記憶體陣列，非持久化；伺服器重啟即失效。
- 訂單與訂閱以 in-memory store (`lib/payment/order-store.ts`，掛在 `globalThis`) 暫存。
- Prisma schema (`Contact` / `Interaction` / `Order` / `Subscription`) 已備妥，待設定 `DATABASE_URL` 並執行 `prisma db push` 後，再將上述 store / seed 切換為 Prisma Client。

技術棧

下表技術自 `package.json` 相依套件與設定檔實際抓取。

類別	技術 / 套件	版本
框架	Next.js (App Router)	16.2.4
UI Runtime	React / React DOM	19.2.4
語言	TypeScript	^5
樣式	Tailwind CSS v4 (@tailwindcss/postcss)	^4
圖示	lucide-react	^1.14.0
UI 工具	class-variance-authority / clsx / tailwind-merge	^0.7.1 / ^2.1.1 / ^3.5.0
ORM	Prisma (prisma + @prisma/client)	^6.19.3

列舉 (enum) :

- `ContactStatus` : LEAD / QUALIFIED / CUSTOMER / CHURNED
- `InteractionType` : CALL / MESSAGE / MEETING / NOTE / EMAIL / LINE
- `OrderProvider` : NEWBPAY
- `OrderStatus` : PENDING / PAID / FAILED / REFUNDED
- `SubscriptionStatus` : ACTIVE / PAST_DUE / CANCELED / EXPIRED

重要目錄結構

```
.
├── app/                                # Next.js App Router
│   ├── layout.tsx                     # 根 layout (字體、metadata、全站 footer 法務連結)
│   ├── page.tsx                       # 首頁: 聯絡人列表 + 4 張統計卡 (讀 seed-data)
│   ├── globals.css                    # Tailwind v4 全域樣式
│   ├── contacts/
│   │   ├── new/page.tsx               # 新增聯絡人表單 (client, fetch POST /api/contacts)
│   │   │   └── [id]/
│   │   │       ├── page.tsx           # 詳細頁 + 互動時間軸 (server component)
│   │   │       ├── InteractionForm.tsx # 新增互動表單 (client, useActionState)
│   │   │       └── actions.ts          # addInteractionAction (Server Action)
│   ├── pricing/page.tsx               # 方案與定價 (讀 TINYCRM_PLANS)
│   ├── payment/success/page.tsx        # 付款結果頁 (讀 query: order / status)
│   ├── terms/page.tsx                 # 服務條款
│   ├── privacy/page.tsx               # 隱私權政策 (個資法第 8 條告知)
│   ├── refund/page.tsx                # 退款政策
│   └── api/
│       ├── contacts/
│       │   ├── route.ts               # POST 新增聯絡人 (demo, 未寫 DB)
│       │   └── export/route.ts         # GET 匯出 Excel (xlsx)
│       ├── lead-summary/route.ts       # POST AI 業務筆記摘要 (Smart Router)
│       └── payment/newebpay/
│           ├── checkout/route.ts       # POST 建立訂單 + MPG 加密參數
│           ├── notify/route.ts         # POST S2S 付款通知 (固定回 "0")
│           └── return/route.ts         # POST 使用者導回 → 303 轉址結果頁
├── lib/
│   ├── company.ts                     # 公司資訊單一來源 (名稱 / 統編 / 地址 / Email)
│   ├── seed-data.ts                  # demo 聯絡人 / 互動 + helper (append / query)
│   ├── utils.ts                       # cn() + 狀態中文標籤 / 徽章樣式
│   ├── router-client.ts               # Smart Router LLM client (chat())
│   └── payment/
│       ├── pricing.ts                 # 方案定價 (basic / pro)
│       ├── order-store.ts             # in-memory 訂單儲存 (鏡像 Prisma Order)
│       └── newebpay.ts                 # MPG 加解密 / TradeSha / buildCheckoutPayload
├── prisma/
│   └── schema.prisma                  # Contact / Interaction / Order / Subscription
├── public/                             # 靜態資源 (預設 Next.js SVG)
├── .env.example                        # 環境變數範例
├── next.config.ts                      # Next.js 設定 (目前為空白預設)
├── eslint.config.mjs                   # ESLint flat config
├── postcss.config.mjs                  # PostCSS (@tailwindcss/postcss)
├── tsconfig.json                       # 路徑別名 @/* → ./
└── package.json
```

設計重點

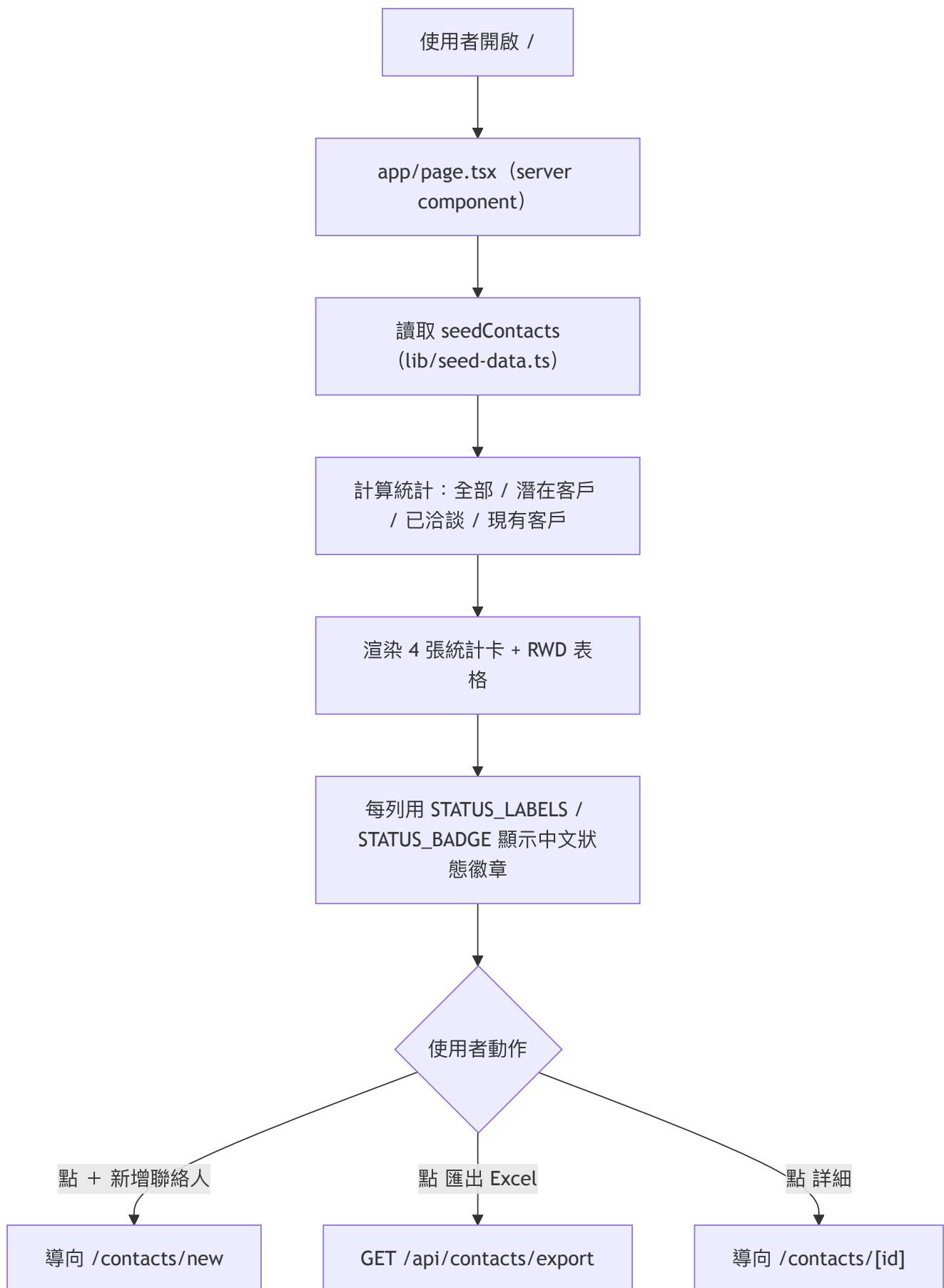
- 單一來源：公司資訊集中於 `lib/company.ts`，方案集中於 `lib/payment/pricing.ts`，全站引用。
- 金流自實作：藍新 MPG 的 AES-256-CBC 加解密與 SHA256 TradeSha 全用 Node `node:crypto` 手寫（`lib/payment/newebpay.ts`），無第三方 SDK；TradeSha 格式為 `HashKey={KEY}&{HEX}&HashIV={IV}`，加密 hex 不加 `TradeInfo=` 前綴。
- AI 走免費層：`/api/lead-summary` 透過 Smart Router 以 `tier: "free"` 呼叫 LLM，避免消耗付費額度；失敗回 502。
- 金流路由標註：`checkout / notify / return` 皆宣告 `runtime = "nodejs"` 與 `dynamic = "force-dynamic"`。

關鍵流程 (FLOWS)

以下流程之節點均對應 repo 內真實的頁面、API Route、Server Action 與函式。各流程旁標註對應檔案。

1. 瀏覽聯絡人列表與統計 (首頁)

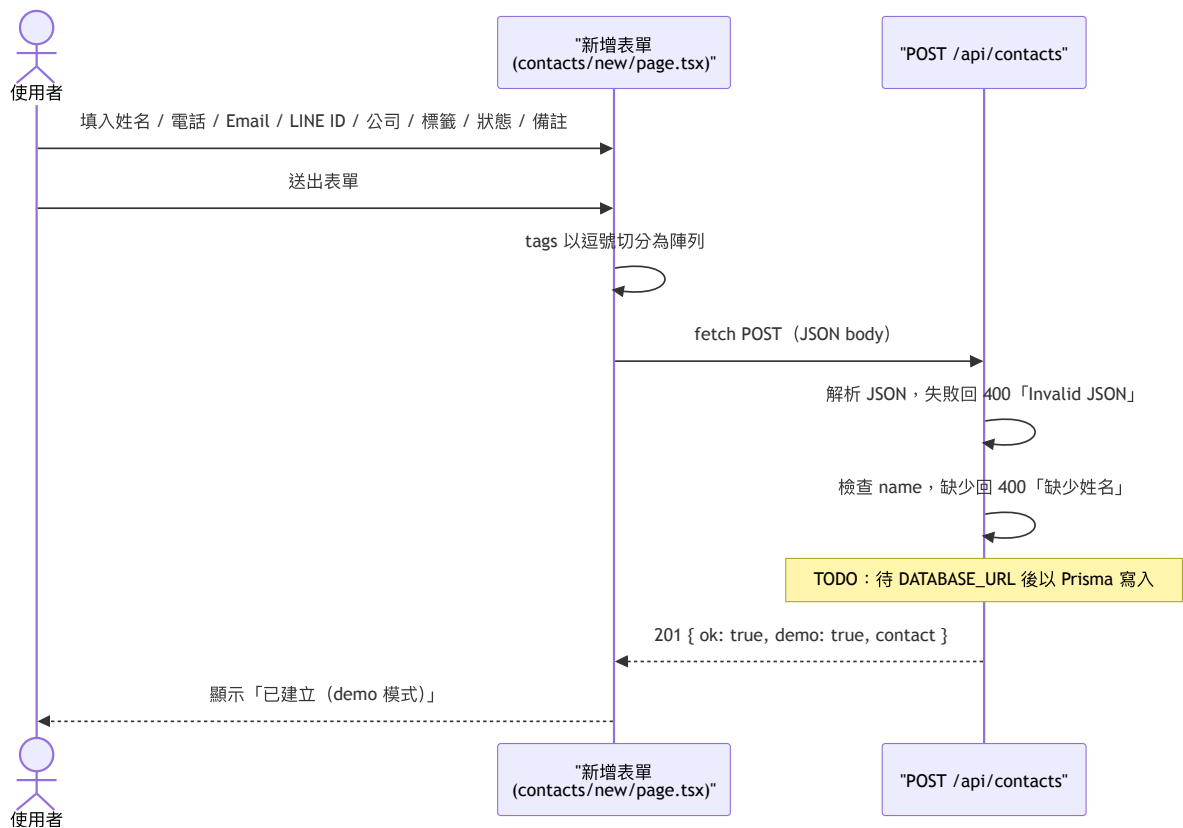
對應： `app/page.tsx`、`lib/seed-data.ts`、`lib/utils.ts`



說明：首頁為 server component，資料目前直接來自 seedContacts (demo 種子)。狀態以中文徽章顯示 (潛在客戶 / 已洽談 / 現有客戶 / 流失)，對應 enum LEAD / QUALIFIED / CUSTOMER / CHURNED。

2. 新增聯絡人 (demo 模式)

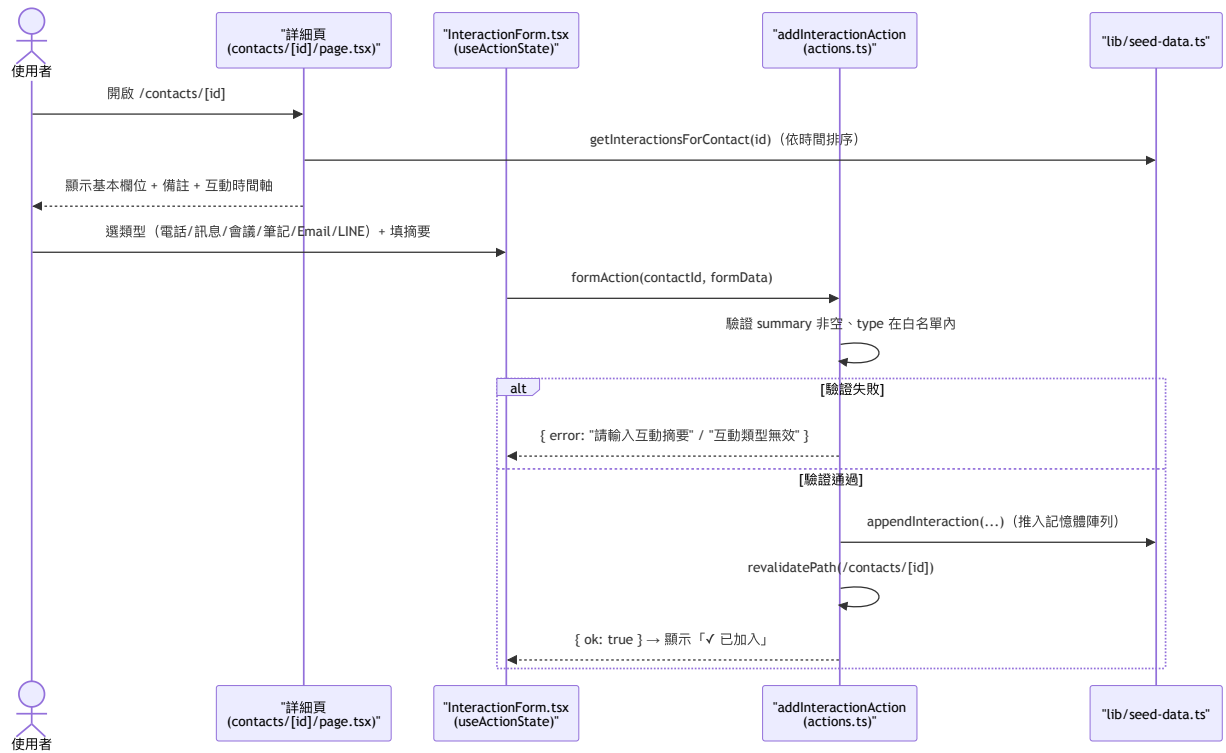
對應： `app/contacts/new/page.tsx` 、 `app/api/contacts/route.ts`



說明：表單為 client component。後端目前不寫入資料庫，僅驗證姓名後回 201 並標 `demo: true`；真正持久化待 Prisma 接線後補上。

3. 新增互動紀錄 (互動時間軸，Server Action)

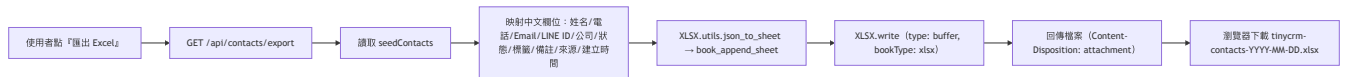
對應： `app/contacts/[id]/page.tsx` 、 `InteractionForm.tsx` 、 `actions.ts` 、 `lib/seed-data.ts`



說明：透過 React `useActionState` 串接 Server Action。新增的互動以 `appendInteraction` 推入 `seedInteractions` 記憶體陣列（非持久化，伺服器重啟即失效），並用 `revalidatePath` 重新渲染時間軸。

4. 匯出聯絡人 Excel

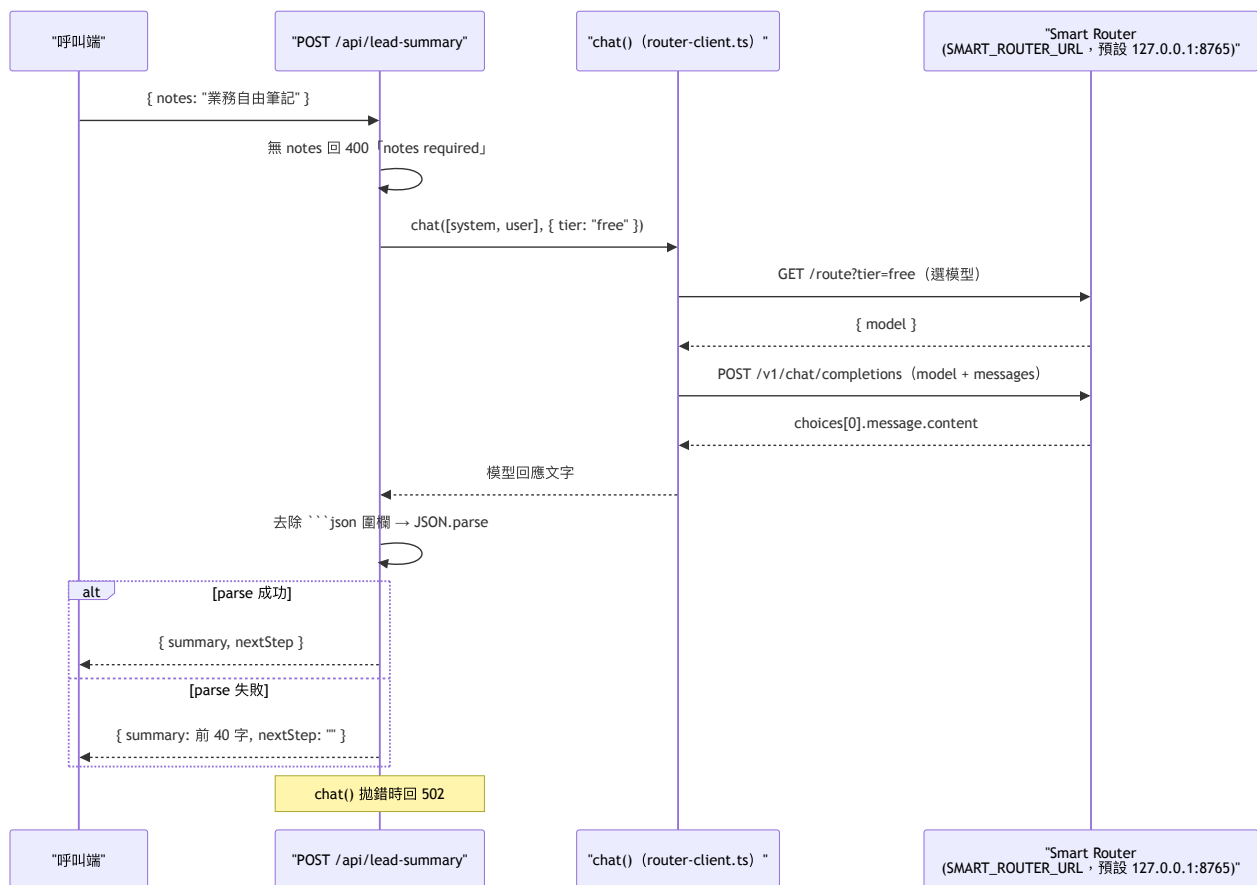
對應： `app/api/contacts/export/route.ts` 、 `lib/seed-data.ts` 、 `xlsx`



說明：使用 SheetJS (`xlsx`) 即時產生 `.xlsx`。狀態欄以 `STATUS_LABELS` 轉中文，標籤以「、」串接，檔名帶當日日期。

5. AI 業務筆記摘要

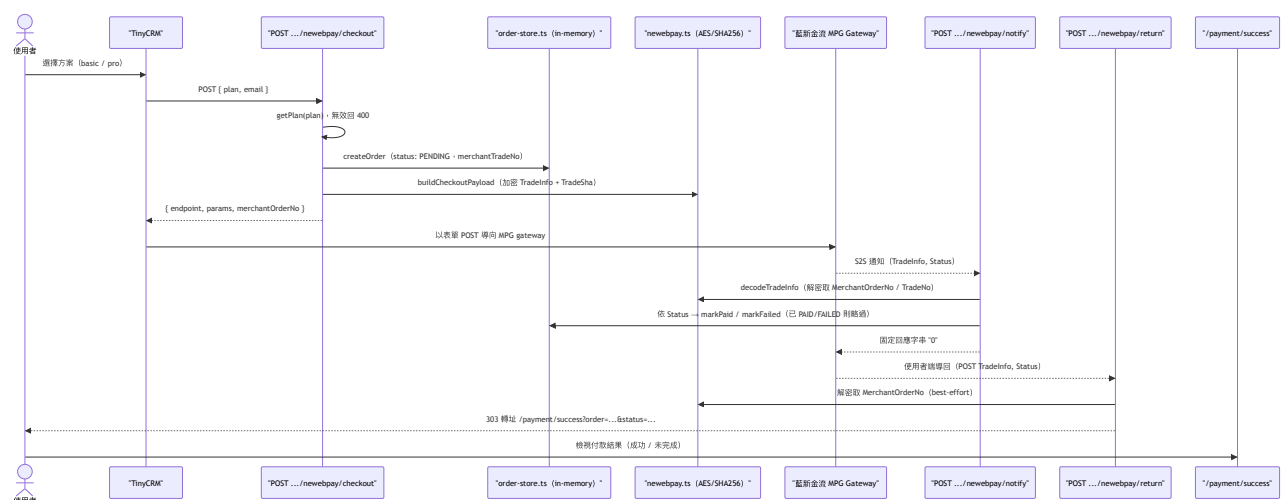
對應： `app/api/lead-summary/route.ts` 、 `lib/router-client.ts` 、 Smart Router



說明：系統提示要求模型輸出 `{ "summary", "nextStep" }` JSON（繁中、字數上限）。走 Smart Router 免費層（`tier: "free"`），避免消耗付費額度；Router 端點異常時 API 回 502。

6. 藍新金流 (NewwebPay) 訂閱付款

對應：checkout/route.ts、notify/route.ts、return/route.ts、lib/payment/*、app/payment/success/page.tsx



說明：

- **checkout**：依方案建立 in-memory 訂單（`PENDING`）並產生 MPG 加密參數（`AES-256-CBC + SHA256 TradeSha`）。

- **notify** : Server-to-server 權威狀態更新，依解密後 `Status === "SUCCESS"` 標記 `markPaid` 否則 `markFailed`，且具幂等保護（訂單已 `PAID` / `FAILED` 則不重複處理），固定回應 `"0"` 避免藍新重送風暴。
- **return** : 僅作使用者端導回，best-effort 解出訂單編號後 303 轉址至 `/payment/success`；狀態變更不在此處進行。
- **success** : 依 query 參數 `order` / `status` 顯示「付款完成」或「付款未完成」與客服資訊。

「PAGES.md」

頁面與 API (PAGES)

路由依 `app/` 實際目錄結構整理。本專案無登入 / 驗證機制，所有頁面與 API 皆公開（無需登入；「需登入」欄一律為「否」）。

頁面 / 路由表 (Web)

路徑	檔案	用途	需登入
<code>/</code>	<code>app/page.tsx</code>	首頁：聯絡人列表 + 4 張統計卡（全部 / 潛在客戶 / 已洽談 / 現有客戶），RWD 表格	否
<code>/contacts/new</code>	<code>app/contacts/new/page.tsx</code>	新增聯絡人表單（姓名 / 電話 / Email / LINE ID / 公司 / 標籤 / 狀態 / 備註），送出至 <code>POST /api/contacts</code>	否
<code>/contacts/[id]</code>	<code>app/contacts/[id]/page.tsx</code>	聯絡人詳細頁 + 互動時間軸；可透過 Server Action 新增互動。查無資料時 <code>notFound()</code>	否
<code>/pricing</code>	<code>app/pricing/page.tsx</code>	方案與定價（讀 <code>TINYCRM_PLANS</code> ）：基礎版 NT\$499/月、進階版 NT\$899/月 + 服務 / 付款說明	否
<code>/payment/success</code>	<code>app/payment/success/page.tsx</code>	付款結果頁，依 query <code>order</code> / <code>status</code> 顯示「付款完成 / 未完成」與客服資訊	否
<code>/terms</code>	<code>app/terms/page.tsx</code>	服務條款	否
<code>/privacy</code>	<code>app/privacy/page.tsx</code>	隱私權政策（依個資法第 8 條告知）	否
<code>/refund</code>	<code>app/refund/page.tsx</code>	退款政策（七日猶豫期之適用與例外）	否

全站 footer（`app/layout.tsx`）固定提供 `/pricing`、`/terms`、`/privacy`、`/refund` 四個法務連結與公司資訊（來源 `lib/company.ts`）。

注意：`/payment/success` 內有「開始使用 TinyCRM」按鈕連到 `/contacts`，但 repo 內並無 `/contacts` 路由（聯絡人列表為首頁 `/`）；此連結目前會導向不存在的頁面。

API Endpoint 表

方法	路徑	檔案	用途
POST	<code>/api/contacts</code>	<code>app/api/contacts/route.ts</code>	新增聯絡人 (demo)。驗證 JSON 與 <code>name</code> ；目前不寫 DB，回 <code>201 { ok, demo, contact }</code>

方法	路徑	檔案	用途
GET	/api/contacts/export	app/api/contacts/export/route.ts	將聯絡人匯出為 .xlsx (SheetJS) , 檔名 tinycrm-contacts-YYYY-MM-DD.xlsx
POST	/api/lead-summary	app/api/lead-summary/route.ts	AI 業務筆記摘要 : body { notes } → 回 { summary, nextStep } ; 經 Smart Router 免費層 ; 失敗回 502
POST	/api/payment/newebpay/checkout	app/api/payment/newebpay/checkout/route.ts	依方案 (basic / pro) 建立訂單並產生 MPG 加密參數 , 回 { endpoint, params, merchantOrderNo }
POST	/api/payment/newebpay/notify	app/api/payment/newebpay/notify/route.ts	藍新 Server-to-server 付款通知 ; 解密驗證後更新訂單狀態 ; 固定回應字串 "0"
POST	/api/payment/newebpay/return	app/api/payment/newebpay/return/route.ts	使用者端導回 ; best-effort 解出訂單編號後 303 轉址至 /payment/success

API 請求 / 回應細節

POST /api/contacts

- Request body (JSON) : name (必填) 、 phone 、 email 、 lineId 、 company 、 tags (string[]) 、 status 、 notes
- 回應 :
 - 400 { error: "Invalid JSON" } (JSON 解析失敗)
 - 400 { error: "缺少姓名" } (缺 name)
 - 201 { ok: true, demo: true, contact } (成功 , 尚未寫入 DB)

POST /api/lead-summary

- Request body (JSON) : { notes: string }
- 回應 :
 - 400 { error: "notes required" }
 - 200 { summary, nextStep }
 - 502 { error } (Smart Router 端點異常)

POST /api/payment/newebpay/checkout

- Request body (JSON) : { plan?: "basic" | "pro", email?: string } (plan 預設 basic)
- 回應 :
 - 400 { error: "invalid plan" }
 - 200 { endpoint, params: { MerchantID, TradeInfo, TradeSha, Version }, merchantOrderNo }

- 前置：需設定 `NEWBPAY_MERCHANT_ID` / `NEWBPAY_HASH_KEY` / `NEWBPAY_HASH_IV`，否則 `buildCheckoutPayload` 拋 `newebpay env not configured`

POST `/api/payment/newebpay/notify`

- Request：藍新以 `multipart/form-data` 傳 `TradeInfo`（加密 hex）、`Status`
- 行為：解密取 `MerchantOrderNo` / `TradeNo`，依 `Status` 標記 `markPaid` / `markFailed`（具冪等保護）
- 回應：一律回 `"0"`（即使例外，避免重送風暴）

POST `/api/payment/newebpay/return`

- Request：藍新以 `form-data` 傳 `TradeInfo` / `Status`
- 回應：303 redirect → `/payment/success?order=...&status=...`

Server Action

名稱	檔案	用途
<code>addInteractionAction(contactId, prev, formData)</code>	<code>app/contacts/[id]/actions.ts</code>	在聯絡人詳細頁新增互動：驗證 <code>summary</code> 非空、 <code>type</code> 屬於 <code>CALL/MESSAGE/MEETING/NOTE/EMAIL/LINE</code> ；以 <code>appendInteraction</code> 寫入記憶體並 <code>revalidatePath</code>

主要共用模組（`lib/`）

模組	重要匯出	說明
<code>lib/company.ts</code>	<code>COMPANY</code>	公司資訊單一來源（ <code>name</code> / <code>legalName</code> / <code>taxId</code> / <code>address</code> / <code>email</code> / <code>lineId</code> ）
<code>lib/seed-data.ts</code>	<code>seedContacts</code> 、 <code>seedInteractions</code> 、 <code>getInteractionsForContact</code> 、 <code>appendInteraction</code> 、 <code>INTERACTION_LABELS</code>	demo 資料與互動 helper
<code>lib/utils.ts</code>	<code>cn</code> 、 <code>STATUS_LABELS</code> 、 <code>STATUS_BADGE</code>	<code>className</code> 合併工具 + 狀態中文標籤 / 徽章樣式
<code>lib/router-client.ts</code>	<code>chat</code> 、 <code>Tier</code>	Smart Router LLM client（OpenAI 相容）
<code>lib/payment/pricing.ts</code>	<code>TINYCRM_PLANS</code> 、 <code>getPlan</code> 、 <code>Plan</code>	方案定價（basic / pro）
<code>lib/payment/order-store.ts</code>	<code>createOrder</code> 、 <code>findByMerchantTradeNo</code> 、 <code>markPaid</code> 、 <code>markFailed</code> 、 <code>listOrders</code> 、 <code>makeMerchantTradeNo</code>	in-memory 訂單儲存（鏡像 Prisma <code>Order</code> ）
<code>lib/payment/newebpay.ts</code>	<code>getNewebpayConfig</code> 、 <code>encodeTradeInfo</code> 、 <code>decodeTradeInfo</code> 、 <code>computeTradeSha</code> 、 <code>buildCheckoutPayload</code>	藍新 MPG 加解密 / <code>TradeSha</code>

方案定價（`lib/payment/pricing.ts`）

code	名稱	價格	內容
<code>basic</code>	TinyCRM 基礎版	NT\$499 / 月	50 位客戶上限 + 預約紀錄 + 客戶分群
<code>pro</code>	TinyCRM 進階版	NT\$899 / 月	無限客戶 + 自動 SMS 提醒 + 消費分析

維運手冊 (OPERATIONS)

環境需求

- Node.js 20+ (@types/node 為 ^20)。
- npm (repo 內含 package-lock.json)。
- (可選) PostgreSQL 連線，用於接上 Prisma 持久化。

安裝與啟動

指令來自 package.json 的 scripts 。

```
# 1. 安裝相依套件
npm install

# 2. 設定環境變數 (見下節)，複製範例後依需求填值
cp .env.example .env

# 3. (可選) 接上資料庫後同步 Prisma schema
npx prisma db push

# 4. 啟動開發伺服器 (預設 http://localhost:3000)
npm run dev
```

指令	說明
npm run dev	啟動 Next.js 開發伺服器 (next dev)
npm run build	正式建置 (next build)
npm run start	啟動正式伺服器 (next start ，需先 build)
npm run lint	執行 ESLint (eslint)

不設定資料庫也能啟動：列表 / 詳細 / 匯出會直接使用 lib/seed-data.ts 的 demo 種子資料；新增聯絡人 / 互動為 demo / 記憶體模式 (不持久化)。

環境變數

以下變數為程式碼 (app/ 、 lib/ 、 prisma/schema.prisma) 實際讀取者。請勿在文件或版控中填入真實值。

變數	必填	讀取位置	說明
DATABASE_URL	接 DB 時必填	prisma/schema.prisma datasource.url	PostgreSQL 連線字串。未設定則以 demo 種子 / in-memory 運作

變數	必填	讀取位置	說明
DATABASE_URL_UNPOOLED	選填	prisma/schema.prisma datasource.directUrl	Prisma directUrl，供 migration / db push 使用的非連線池直連 (Neon 建議設定)
NEXT_PUBLIC_SITE_URL	建議	checkout/route.ts、 return/route.ts	站台公開網址，用於組合金流 return / notify / clientBack URL；未設定時 fallback 為 req.nextUrl.origin。本機建議 http://localhost:3000
NEWBPAY_MERCHANT_ID	啟用 金流 時必 填	lib/payment/newebpay.ts	藍新金流商店代號
NEWBPAY_HASH_KEY	啟用 金流 時必 填	lib/payment/newebpay.ts	藍新 HashKey (AES-256-CBC 金鑰 / SHA256)
NEWBPAY_HASH_IV	啟用 金流 時必 填	lib/payment/newebpay.ts	藍新 HashIV (AES IV)
NEWBPAY_API_BASE	選填	lib/payment/newebpay.ts	藍新 API base，預設 https://ccore.newebpay.com (測試環境)。正式 環境改為 https://core.newebpay.com
SMART_ROUTER_URL	選填	lib/router-client.ts	Smart Router 端點，供 /api/lead-summary 呼叫 LLM，預設 http://127.0.0.1:8765

.env.example 目前列有 DATABASE_URL、DATABASE_URL_UNPOOLED、NEWBPAY_MERCHANT_ID、NEWBPAY_HASH_KEY、NEWBPAY_HASH_IV、NEWBPAY_API_BASE、SMART_ROUTER_URL、NEXT_PUBLIC_SITE_URL，與上表一致。

資料庫接線 (Prisma)

目前資料層為 demo / in-memory，要切換為持久化：

- 準備 PostgreSQL (如 Neon / Supabase)，取得連線字串。
- 設定 DATABASE_URL (與 Neon 建議的 DATABASE_URL_UNPOOLED)。
- 建立資料表結構：

```
npx prisma db push      # 直接同步 schema (開發 / 首次建表)
# 或
npx prisma migrate dev  # 以 migration 管理 (需互動環境)
npx prisma generate     # 產生 Prisma Client
```

- 將下列以 seed / in-memory 實作的模組改用 Prisma Client：
 - lib/seed-data.ts (Contact / Interaction 讀寫)
 - app/api/contacts/route.ts (目前 TODO: persist via Prisma)
 - app/contacts/[id]/actions.ts 的 appendInteraction

- `lib/payment/order-store.ts` (`Order` ; in-memory store 已刻意鏡像 Prisma `Order` 欄位，API surface 穩定可直接替換)

部署 (建議 Vercel)

1. 將 repo 連接至 Vercel (GitHub → Import Project)。
2. 在 **Settings → Environment Variables** 設定上節變數 (至少 `DATABASE_URL` ; 啟用金流再加 `NEWBYPAY_*` 與 `NEXT_PUBLIC_SITE_URL`)。
3. 資料庫建議使用 Neon Postgres (或 Supabase) ; 於 build 前以 `prisma db push` 或 migration 建表。
4. 推送至預設分支即觸發部署。

注意事項：

- `/api/lead-summary` 依賴 Smart Router (`SMART_ROUTER_URL` , 預設指向 `127.0.0.1:8765` 本機)。部署到雲端後須改指向可公開存取的相容端點，否則該 API 會回 502。
- in-memory 訂單 store (`order-store.ts`) 掛在 `globalThis` , 在 serverless / 多實例環境下不可靠 (跨請求 / 跨實例不共享、冷啟動會清空)。正式金流務必先完成 Prisma `Order` 接線。
- 金流相關路由已標 `runtime = "nodejs"` 與 `dynamic = "force-dynamic"`。

常見維運操作

情境	操作
查看 / 編輯資料 (接 DB 後)	<code>npx prisma studio</code>
修改方案 / 價格	編輯 <code>lib/payment/pricing.ts</code> (<code>TINYCRM_PLANS</code>) , 全站定價 / checkout 自動套用
更新公司資訊 (名稱 / 統編 / 地址 / Email / LINE)	編輯 <code>lib/company.ts</code> (<code>COMPANY</code>) , footer 與法務頁全站生效
調整 demo 種子資料	編輯 <code>lib/seed-data.ts</code>
切換藍新測試 / 正式環境	設定 <code>NEWBYPAY_API_BASE</code> (測試 <code>ccore</code> 、正式 <code>core</code>)
程式碼風格檢查	<code>npm run lint</code>
本機驗證正式建置	<code>npm run build && npm run start</code>

金流串接備忘 (藍新 NewePay MPG)

- 加密：AES-256-CBC (PKCS7, block 32), 輸出 hex (`encodeTradeInfo`)。
- 驗章：SHA256, 格式 `HashKey={KEY}&{HEX}&HashIV={IV}` , 輸出大寫 hex ; 加密 hex 不加 `TradeInfo=` 前綴 (`computeTradeSha`)。
- checkout 送出的固定欄位： `Version: "2.0"`、 `RespondType: "JSON"`、 `ItemDesc` 截斷 50 字。
- `notify` 須永遠回應字串 `"0"` (含例外情況), 否則藍新會觸發重送風暴。
- `merchantTradeNo` 由 `makeMerchantTradeNo("TCRM")` 產生並截斷為 20 字 (藍新上限 30 字)。

待辦 / 已知限制

- `POST /api/contacts` 與新增互動為 demo / 記憶體模式, 尚未寫入資料庫。
- 訂單 / 訂閱以 in-memory store 暫存, 未接 Prisma。
- `Subscription` 模型已定義, 但目前程式碼未有任何讀寫該表的邏輯。

- `lib/company.ts` 的 `taxId` 與 `lineId` 為待補佔位字串 (「(統編待補)」 / 「(LINE 官方帳號待補)」)。
- `/payment/success` 的「開始使用」按鈕連向 `/contacts`，但該路由不存在 (列表在首頁 /)。
- 無使用者驗證 / 登入機制；所有頁面與 API 皆公開。